



WILLSIM

FICHE DE COURS · RESSOURCE PÉDAGOGIQUE GRATUITE

Vision par ordinateur pour la robotique

Repérer une pièce avec une caméra USB, mesurer sa position et son orientation, puis convertir ces pixels en coordonnées utilisables par un bras robotisé : les bases de la vision industrielle avec Python et OpenCV, et une ouverture vers la détection d'objets par réseau de neurones.

NIVEAU

Intermédiaire

DURÉE

8 h (4 séances de 2 h)

PRÉREQUIS

Python et notions de NumPy

OUTILS

Python, OpenCV, NumPy, caméra USB

OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- isoler un objet coloré par seuillage dans l'espace HSV et nettoyer le masque obtenu ;
- calculer le centroïde et l'orientation d'un objet à partir des moments de son contour, à 1 pixel et 1 degré près sur une image de test ;
- calibrer une caméra avec un damier et interpréter la matrice intrinsèque et l'erreur de reprojection ;
- convertir des pixels en millimètres sur le plan de travail par homographie, et contrôler le résultat sur un repère ;
- décrire la chaîne « détecter, localiser, transmettre » d'un pick-and-place, en citer les principales sources d'erreur et situer les détecteurs neuronaux de la famille YOLO.

1. Représentation d'une image

Pour OpenCV, une image couleur est un tableau NumPy de forme (hauteur, largeur, 3) et de type `uint8` : chaque pixel contient trois intensités entières de 0 à 255. Deux conventions surprennent souvent :

- les canaux sont rangés dans l'ordre **B, G, R** (bleu, vert, rouge) : une image lue par `cv2.imread` ou `cv2.VideoCapture` doit être convertie avant d'être affichée par une bibliothèque qui attend du RVB, comme Matplotlib ;
- l'origine est le **coin supérieur gauche** ; l'axe des colonnes u (x dans la documentation d'OpenCV) va vers la droite, l'axe des lignes v (y) vers le bas. On lit un pixel avec `img[v, u]` : la ligne d'abord.

Une image 640 × 480 en BGR occupe 921 600 octets. Comme le temps de calcul croît à peu près avec le nombre de pixels, choisissez la plus petite résolution qui suffit à la précision visée.

2. Espaces colorimétriques et seuillage HSV

En BGR, couleur et luminosité sont mêlées : une même pièce rouge, à l'ombre ou au soleil, a des valeurs B, G, R très différentes. L'espace **HSV** les sépare : **H** (teinte) est un angle sur le cercle chromatique, qu'OpenCV divise par 2 pour les images 8 bits afin qu'il tienne dans [0 ; 179] ; **S** (saturation) va de 0 (gris) à 255 (couleur vive) ; **V** (valeur, la luminosité) de 0 (noir) à 255.

Le seuillage garde les pixels dont H, S et V sont compris entre deux bornes : `cv2.inRange` renvoie un **masque binaire** (255 si le pixel est retenu, 0 sinon). Les bornes basses de S et V écartent le fond grisâtre et les zones sombres, où la teinte n'a plus de sens. Le rouge, à cheval sur 0, demande deux appels à `cv2.inRange` réunis par `cv2.bitwise_or`. Une **ouverture** morphologique (érosion puis dilatation) supprime ensuite les pixels isolés, et une **fermeture** (dilatation puis érosion) bouche les petits trous.

Plages de teinte indicatives dans OpenCV, à ajuster sous votre éclairage

Couleur	Rouge	Orange	Jaune	Vert	Bleu
Teinte	0° (360°)	≈ 30°	60°	120°	240°
H dans OpenCV	0 à 8 et 170 à 179	8 à 22	22 à 35	35 à 85	95 à 130

Le programme de test fabrique sa propre image, ce qui permet de comparer le résultat à la vérité terrain. Sa seconde partie figure à la section suivante.

detection_piece.py – 1re partie : image de test et seuillage

```
import cv2
import numpy as np

# Image de test 640 x 480 : fond gris, pièce orange inclinée de 30°, bruit gaussien
img = np.full((480, 640, 3), 90, np.uint8)
coins = cv2.boxPoints(((400, 220), (160, 60), 30)).astype(np.int32) # centre, taille, angle
cv2.fillPoly(img, [coins], (0, 140, 255)) # orange, en BGR
bruit = np.random.default_rng(0).normal(0, 8, img.shape)
img = np.clip(img + bruit, 0, 255).astype(np.uint8)

# 1. Seuillage HSV (images 8 bits : H dans [0, 179], S et V dans [0, 255])
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
masque = cv2.inRange(hsv, (8, 120, 120), (22, 255, 255))
noyau = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
masque = cv2.morphologyEx(masque, cv2.MORPH_OPEN, noyau) # ôte les points isolés
```

3. Contours, centroïde et orientation

`cv2.findContours` suit la frontière de chaque zone blanche du masque et renvoie une liste de polygones ; avec `cv2.RETR_EXTERNAL`, seuls les contours extérieurs sont gardés, et l'on écarte les petites taches par leur aire (`cv2.contourArea`). Les **moments** d'un contour résument sa géométrie : m_{00} est son aire, les moments d'ordre 1 donnent son centroïde $(\bar{u}, \bar{v})$, et les moments centrés d'ordre 2 $(\mu_{20}, \mu_{02}, \mu_{11})$, analogues à un tenseur d'inertie, donnent l'angle θ entre l'axe u et l'axe principal (« long ») de la pièce.

$$\bar{u} = \frac{m_{10}}{m_{00}} \quad \bar{v} = \frac{m_{01}}{m_{00}} \quad \theta = \frac{1}{2} \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$$

- L'axe v étant orienté vers le bas, un angle positif est une rotation dans le **sens horaire** à l'écran.
- θ définit un axe, pas un sens : il est connu **à 180° près**, ce qui suffit pour une pince à deux mors, pas pour une pièce qui a un « avant ».

- Pour un disque, un carré ou un polygone régulier, $\mu_{20} = \mu_{02}$ et $\mu_{11} = 0$ quelle que soit la rotation : θ n'a plus de sens. Pour un carré, `cv2.minAreaRect` donne une orientation à 90° près ; sa convention d'angle a changé selon les versions d'OpenCV, vérifiez-la sur une image de test.

detection_piece.py – 2e partie, à la suite de la première : contour et moments

```
# 2. Contours extérieurs ; on garde le plus grand
contours, _ = cv2.findContours(masque, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if not contours:
    raise SystemExit("Aucune pièce détectée : revoir les seuils HSV")
piece = max(contours, key=cv2.contourArea)

# 3. Centroïde et orientation de l'axe principal, par les moments
m = cv2.moments(piece)
u, v = m["m10"] / m["m00"], m["m01"] / m["m00"]
theta = 0.5 * np.degrees(np.arctan2(2 * m["mu11"], m["mu20"] - m["mu02"]))
print(f"centre = ({u:.1f}, {v:.1f}) px ; orientation = {theta:.1f}°")
```

Résultat affiché : centre = (399.5, 219.5) px ; orientation = 29.8° pour une pièce dessinée en (400, 220) à 30° ; le demi-pixel d'écart vient des sommets du dessin, tronqués à l'entier. Programmes de la fiche testés avec Python 3.8 (NumPy 1.24, OpenCV 4.5) et Python 3.9 (NumPy 2.0, OpenCV 4.13).

4. Calibration de caméra avec un damier

Le **modèle sténopé** (*pinhole*) relie un point (X, Y, Z) du repère d'un objet à son pixel (u, v) :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R | t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad \text{avec} \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

La **matrice intrinsèque** K contient les focales f_x, f_y (en pixels) et le centre optique (c_x, c_y) ; R et t (paramètres extrinsèques) placent l'objet par rapport à la caméra ; s est un facteur d'échelle. Un objectif réel ajoute une **distorsion**, surtout radiale (les droites se courbent vers les bords), décrite par les coefficients k_1, k_2, p_1, p_2, k_3 .

Pour estimer K et la distorsion, on photographie une quinzaine de fois un damier collé sur un support plan et rigide, en variant position, inclinaison et distance, bords de l'image compris. On mesure une case au pied à coulisse, car l'impression peut la modifier. `cv2.findChessboardCorners` détecte les **coins intérieurs** (9 × 6 pour 10 × 7 cases), `cv2.cornerSubPix` les affine et `cv2.calibrateCamera` renvoie l'erreur de reprojection RMS, K et la distorsion. Ci-dessous, des vues synthétiques produites par une caméra connue remplacent les photos : on vérifie que la calibration la retrouve. Avec de vraies photos, lisez chaque vue avec `cv2.imread` et convertissez-la en niveaux de gris.

calibration_synthetique.py – 1re partie : damier virtuel

```
import cv2
import numpy as np

# Damier virtuel de 10 x 7 cases de 25 mm (1 px = 1 mm) : 9 x 6 coins intérieurs
NX, NY, CASE = 9, 6, 25
cases = np.indices((NY + 1, NX + 1)).sum(axis=0) % 2
damier = (np.kron(cases, np.ones((CASE, CASE))) * 255).astype(np.uint8)
damier = cv2.copyMakeBorder(damier, CASE, CASE, CASE, CASE, cv2.BORDER_CONSTANT, value=255)
centre = np.array([damier.shape[1] / 2, damier.shape[0] / 2, 0])
objp = np.zeros((NX * NY, 3), np.float32) # coins dans le plan Z = 0, en mm
objp[:, :2] = np.mgrid[0:NX, 0:NY].T.reshape(-1, 2) * CASE
```

```

K_vrai = np.array([[800.0, 0, 320], [0, 800, 240], [0, 0, 1]]) # inconnue en pratique
rng = np.random.default_rng(1)
critere = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.01)
pts_3d, pts_2d = [], []
for _ in range(15):
    # 15 poses aléatoires du damier
    R, _ = cv2.Rodrigues(rng.uniform(-0.35, 0.35, 3))
    t = np.array([0, 0, rng.uniform(550, 750)]) - R @ centre
    H = K_vrai @ np.column_stack((R[:, 0], R[:, 1], t)) # plan du damier -> image
    vue = cv2.warpPerspective(damier, H, (640, 480), borderValue=170)
    ok, coins = cv2.findChessboardCorners(vue, (NX, NY))
    if ok:
        pts_2d.append(cv2.cornerSubPix(vue, coins, (5, 5), (-1, -1), critere))
        pts_3d.append(objp)

rms, K, dist, _, _ = cv2.calibrateCamera(pts_3d, pts_2d, (640, 480), None, None)
print(f"{len(pts_2d)} vues ; erreur de reprojection RMS = {rms:.3f} px")
print(np.round(K, 1))

```

Résultat affiché : 15 vues, erreur RMS d'environ 0,05 px, focales estimées à 799,4 px pour 800 et centre optique retrouvé à 0,1 px près. Sur de vraies photos, quelques dixièmes de pixel sont un bon résultat ; au-delà d'un pixel, cherchez une image floue, un damier gondolé ou une taille de case erronée. La calibration ne vaut que pour une résolution et une mise au point données : désactivez l'autofocus si possible.

5. Des pixels aux millimètres : l'homographie

Quand les pièces reposent sur une table plane, les pixels (u, v) d'une image corrigée de sa distorsion et les points (X, Y) de la table sont liés par une **homographie** H , matrice 3×3 définie à un facteur près (8 degrés de liberté) :

$$s \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Quatre correspondances, sans trois points alignés, suffisent à la calculer ; avec davantage de points,

`cv2.findHomography` donne une solution aux moindres carrés et peut écarter les points aberrants

(`cv2.RANSAC`). Astuce : relevez les repères **directement dans le repère du robot**, en y amenant la pointe de la pince ; H donne alors des coordonnées robot.

pixels_vers_robot.py

```

import cv2
import numpy as np

# Quatre repères collés sur la table : position mesurée dans l'image (px)...
px = np.float32([[165, 426], [578, 444], [573, 142], [186, 133]])
# ... et dans le repère du robot (mm), relevée en y amenant la pointe de la pince
mm = np.float32([[150, 200], [150, -200], [450, -200], [450, 200]])
H, _ = cv2.findHomography(px, mm) # 4 points au minimum ; davantage = plus robuste

def pixel_vers_robot(u, v):
    """Convertit un pixel (u, v) en coordonnées (X, Y) en mm sur le plan de la table."""
    p = np.float32([[u, v]]) # forme (N, 1, 2) attendue par OpenCV
    return cv2.perspectiveTransform(p, H)[0, 0]

# Contrôle sur un 5e repère, non utilisé pour le calcul : attendu (380, -120) mm
X, Y = pixel_vers_robot(494, 207)
print(f"repère de contrôle : X = {X:.1f} mm ; Y = {Y:.1f} mm")

```

Résultat affiché : `X = 379.7 mm ; Y = -120.1 mm` pour un repère placé en (380, -120) mm ; l'écart vient de l'arrondi des pixels des repères. Si la distorsion est visible, corrigez d'abord les pixels des repères et des pièces avec

```
cv2.undistortPoints(points, K, dist, P=K).
```

5.1 Limite : les objets ont une hauteur

H n'est exacte que dans le plan qui a servi à la calculer. Le dessus d'une pièce de hauteur h est vu là où son rayon de vue coupe la table, donc plus loin de l'aplomb de la caméra. Pour une caméra verticale à la hauteur Z_c , un point situé à la distance horizontale r de cet aplomb est décalé de $e = r \cdot h / (Z_c - h)$. Avec $Z_c = 800$ mm, $h = 30$ mm et $r = 200$ mm, $e \approx 7,8$ mm : assez pour manquer la prise. Remèdes : calculer H avec des repères posés à la hauteur du dessus des pièces, ou multiplier la distance du point mesuré à l'aplomb de la caméra par $(Z_c - h) / Z_c$.

À RETENIR

- Une image OpenCV est un tableau NumPy (hauteur, largeur, 3) en BGR, lu par `img[v, u]` ; en HSV, H va de 0 à 179.
- Centroïde = $(m_{10}/m_{00}, m_{01}/m_{00})$; orientation = $\frac{1}{2} \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$, connue à 180° près.
- La calibration se fait une fois par caméra, résolution et mise au point ; l'homographie, une fois par installation, à refaire dès que la caméra bouge.
- L'homographie n'est exacte que dans le plan qui a servi à la calculer.

6. Chaîne pick-and-place : détecter, localiser, transmettre

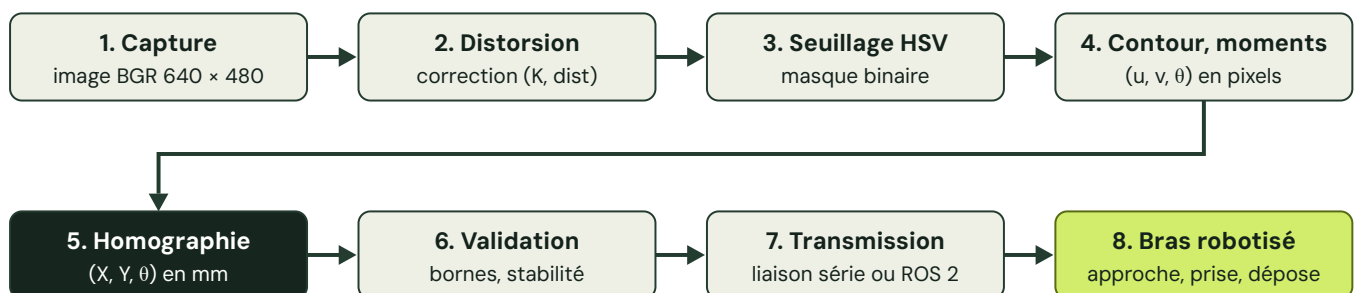


Figure 1 — Chaîne de traitement d'un pick-and-place guidé par caméra : étapes 1 à 4 sur l'image, 5 à 8 dans le repère du robot.

À chaque cycle, la vision produit une **cible** (X, Y, θ) dans le repère du robot. Une homographie ne conservant pas les angles, on convertit θ en transformant deux points de l'axe de la pièce (le centroïde et un point situé une vingtaine de pixels plus loin dans la direction θ), puis en recalculant l'angle entre les deux points obtenus, de l'axe X du robot vers l'axe Y. Avant tout envoi, on **valide** la cible : atteignable, zone dégagée, aire plausible, détection stable sur plusieurs images.

La cible part ensuite vers le bras, par liaison série vers le microcontrôleur qui pilote les servomoteurs ou sur un *topic* ROS 2, dans un format explicite : par exemple, la ligne `312.4; -41.0; 27.5` transmet X et Y en millimètres et θ en degrés, et le microcontrôleur répond `OK` après avoir vérifié les bornes, `ERR` sinon. La prise suit une séquence prudente : approche à hauteur de sécurité, descente lente, fermeture de la pince, remontée, dépose. Prenez l'image quand le bras est hors du champ.

ATTENTION · SÉCURITÉ

- Dégagez la zone de travail et gardez à portée de main un arrêt d'urgence ou l'interrupteur des moteurs ; coupez-le avant toute intervention mécanique ou tout recâblage.
- Alimentez les servomoteurs par une alimentation dédiée, dimensionnée pour leur courant de blocage, jamais par le port USB ni par le régulateur 5 V de la carte ; reliez les masses.
- Testez toute nouvelle cible à vitesse réduite et à hauteur de sécurité, mains hors de la zone ; vérifiez aussi les bornes côté microcontrôleur, au cas où la vision se tromperait.

EXERCICE GUIDÉ · DE L'IMAGE À LA CONSIGNE DU BRAS

En réunissant `detection_piece.py` et `pixels_vers_robot.py`, écrivez `localiser(img)`, qui renvoie la cible (X, Y, θ) dans le repère du robot, ou `None` si aucune pièce n'est visible.

1. Transformez `detection_piece.py` en une fonction `detecter(img)` qui ignore les contours de moins de 500 px² et renvoie `None` s'il n'en reste aucun.
2. Générez cinq images de test : la même pièce de 160 × 60 px en (250, 300) à 0°, en (400, 200) à 45°, en (350, 330) à -60° et en (100, 300) à 0°, puis une image sans pièce.
3. Convertissez le centroïde avec `pixel_vers_robot`, puis l'orientation par la méthode des deux points, ramenée dans $[-90^\circ; 90^\circ]$.
4. Construisez la trame $x;y;\theta$ avec une décimale et refusez toute cible hors de la zone $150 \leq X \leq 450 \text{ mm}$, $-200 \leq Y \leq 200 \text{ mm}$.

Critères de réussite : centroïde à moins de 1 px et orientation à moins de 1° des valeurs imposées (angles comparés à 180° près) ; aucune exception sur l'image vide ; trames égales, à ± 1 près sur chaque valeur, à 276.9;122.8;-88.0, 384.8;-24.7;45.7 et 251.0;21.9;-28.0 ; cible de la pièce en (100, 300), où $Y \approx 276 \text{ mm}$, refusée.

Indice : $(a - b + 90) \% 180 - 90$ ramène l'écart entre deux angles connus à 180° près dans $[-90^\circ; 90^\circ]$.

7. Introduction à la détection d'objets par apprentissage profond

Le seuillage HSV atteint ses limites dès que les pièces ne se distinguent plus par leur couleur, que le fond est encombré ou que plusieurs types d'objets se mélangent. Les détecteurs à **réseaux de neurones convolutifs** apprennent alors à reconnaître les objets sur des images annotées. Pour chaque objet, ils renvoient une **boîte englobante**, une **classe** et un **score de confiance** entre 0 et 1 ; une étape de suppression des non-maxima (NMS) élimine en général les doublons. La famille **YOLO** (*You Only Look Once*, J. Redmon et coll., 2015) prédit toutes les boîtes en un seul passage du réseau, d'où son usage en temps réel ; de nombreuses versions en dérivent, dont celles d'Ultralytics.

Une boîte alignée sur les axes de l'image ne donne ni l'orientation ni le contour exact : on applique alors les sections 2 et 3 dans la boîte, ou l'on choisit un modèle de segmentation ou de boîtes orientées (OBB), puis la chaîne continue à l'identique. Ces modèles tournent par exemple avec le module `dnn` d'OpenCV (format ONNX), après un affinage (*fine-tuning*) sur des images prises dans les conditions réelles de l'atelier.

En résumé : le seuillage ne demande aucune donnée d'apprentissage, tourne en quelques millisecondes sur un PC en 640 × 480 et atteint le sous-pixel sur une scène maîtrisée, mais reste sensible au fond et à l'éclairage. Un détecteur neuronal exige souvent plusieurs centaines d'images annotées par classe et davantage de calcul (un GPU ou un NPU aide en temps réel), mais tolère bien mieux les scènes variées.

8. Pièges courants : éclairage, reflets, latence

Symptômes fréquents, causes probables et remèdes

Symptôme	Cause probable	Remède
Seuils instables au fil de la journée, ombres détectées avec la pièce	lumière du jour, ombres portées, exposition et balance des blancs automatiques	éclairage artificiel diffus, constant, venant de plusieurs côtés ; caméra en réglages manuels
Trous dans le masque d'une pièce brillante	reflets spéculaires : V proche de 255, S faible	éclairage diffus ou indirect, filtre polarisant, fermeture morphologique
Contours flous, centroïde instable	flou de bougé (exposition longue) ou autofocus	plus de lumière pour raccourcir l'exposition ; mise au point fixe
Erreur croissante vers les bords	distorsion non corrigée, ou caméra déplacée depuis le calcul de H	calibrer ; refaire H ; contrôler un repère à chaque démarrage

La **latence** mérite un calcul : une pièce qui défile à 100 mm/s, avec 150 ms de retard entre la prise de vue et l'action du bras, est déjà 15 mm plus loin que la position mesurée. Mesurez ce retard, horodatez les images et compensez. Source souvent ignorée : `cv2.VideoCapture` garde des images en tampon, et `read()` peut renvoyer une image ancienne si le traitement est plus lent que la caméra ; réduisez ce tampon (`cv2.CAP_PROP_BUFFERSIZE`, que seuls certains pilotes acceptent) ou lisez les images en continu dans un fil d'exécution séparé.

Pour aller plus loin

- [OpenCV : espaces de couleur](https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html) — https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
- [OpenCV : caractéristiques des contours \(moments\)](https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html) — https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html
- [OpenCV : calibration de caméra](https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html) — https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
- [OpenCV : homographie](https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html) — https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html
- [Documentation de NumPy](https://numpy.org/doc/stable/) — <https://numpy.org/doc/stable/>
- [Article fondateur de YOLO](https://arxiv.org/abs/1506.02640) — <https://arxiv.org/abs/1506.02640>
- [Documentation Ultralytics YOLO](https://docs.ultralytics.com/) — <https://docs.ultralytics.com/>
- [Page du cours sur willsim.fr](https://willsim.fr/cours/vision-robotique-opencv.html) — <https://willsim.fr/cours/vision-robotique-opencv.html>

Fiche rédigée par l'équipe pédagogique WILLSIM, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécanique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 (creativecommons.org/licenses/by-nc-sa/4.0/deed.fr) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à contact@willsim.fr · willsim.fr